



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Hashtables – Collision Resolution



Collision Resolution

- ▶ Perfect hash functions are hard to come by, especially if you do not know the input keys beforehand.
- ▶ You will always need a systematic way of handling collisions.
 - ▶ This is referred to as collision resolution
- ▶ One method is to systematically find an empty slot in the table and put the value in this slot. This technique is called 'open addressing'. For example, start at the original hash value position (slot), look sequentially until you find a slot which is empty.

"open addressing" refers to the fact that the location ("address") of the item is not determined by its hash value.



Collision Resolution: Open addressing

- ▶ In **open addressing**, when a collision occurs, we continue to probe subsequent index positions in the table for a free slot
 - ▶ The sequence of locations that are examined is the *probe sequence*
- ▶ **Linear probing**
 - ▶ Search the hash table **sequentially**, starting from the original location given by the hash function
 - ▶ Possible problem: primary clustering
- ▶ **Quadratic probing**
 - ▶ Search the hash table, trying **increments** of 1^2 , 2^2 , 3^2 and so on, always starting from the original location given by the hash function
 - ▶ Possible problem: secondary clustering
- ▶ **Double hashing**
 - ▶ Use **two hash functions** – if the main hash function generates a collision, then start from that location and consider every n^{th} location, where n is determined by a second hash function



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example:

Insert 44 $h(key) = key \% 13$

$$44 \% 13 = 5 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example:

Insert 44 $h(key) = key \% 13$

$$44 \% 13 = 5 \text{ (collision)}$$

$$(5 + 1) \% 13 = 6 \text{ available}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example: **Insert 44** $h(key) = key \% 13$

$$44 \% 13 = 5 \text{ (collision)}$$

$$(5 + 1) \% 13 = 6 \text{ available}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example:

Insert 51 $h(key) = key \% 13$

$$51 \% 13 = 12 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example: **Insert 51** $h(key) = key \% 13$

$$51 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example: **Insert 51** $h(key) = key \% 13$

$$51 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 2) \% 13 = 1 = \text{available}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

► Example: **Insert 51** $h(key) = key \% 13$

$$51 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 2) \% 13 = 1 = \text{available}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	51	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Linear probing

► Properties

- Simplest way to resolve a collision
- Starting from the collision index, look sequentially until you find an empty slot (search step by step with a step size of 1)
- Wrap around to the start of the table when the end is reached

Clustering occurs when regions of the table become full and there are long runs of filled slots

0	1	2	3	4	5	6	7	8	9	10	11	12
26	51	54	94	17	31	44	None	None	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 38** $h(key) = key \% 13$

$$38 \% 13 = 12 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 38** $h(key) = key \% 13$

$$38 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 38** $h(key) = key \% 13$

$$38 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 38** $h(key) = key \% 13$

$$38 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

$$(12 + 9) \% 13 = 8 \Rightarrow \text{available}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 38** $h(key) = key \% 13$

$$38 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

$$(12 + 9) \% 13 = 8 \text{ (available)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 25** $h(key) = key \% 13$

$$25 \% 13 = 12 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 25** $h(key) = key \% 13$

$$25 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps

- ▶ Example: **Insert 25** $h(key) = key \% 13$

$$25 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 25** $h(key) = key \% 13$

$$25 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

$$(12 + 9) \% 13 = 8 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps
- ▶ Example: **Insert 25** $h(key) = key \% 13$

$$25 \% 13 = 12 \text{ (collision)}$$

$$(12 + 1) \% 13 = 0 \text{ (collision)}$$

$$(12 + 4) \% 13 = 3 \text{ (collision)}$$

$$(12 + 9) \% 13 = 8 \text{ (collision)}$$

$$(12 + 16) \% 13 = 2 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps

▶ Example: **Insert 25** $h(key) = key \% 13$

$25 \% 13 = 12$ (collision)

$(12 + 1) \% 13 = 0$ (collision)

$(12 + 4) \% 13 = 3$ (collision)

$(12 + 9) \% 13 = 8$ (collision)

$(12 + 16) \% 13 = 2$ (collision)

$(12 + 25) \% 13 = 11 \Rightarrow$ available

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	None	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps

▶ Example: **Insert 25** $h(key) = key \% 13$

$25 \% 13 = 12$ (collision)

$(12 + 1) \% 13 = 0$ (collision)

$(12 + 4) \% 13 = 3$ (collision)

$(12 + 9) \% 13 = 8$ (collision)

$(12 + 16) \% 13 = 2$ (collision)

$(12 + 25) \% 13 = 11 \Rightarrow$ available

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	25	77



Open Addressing – Quadratic probing

- ▶ Designed to avoid primary clustering
 - ▶ When a collision occurs, we advance the search for an empty position a considerable **distance** from the collision point
 - ▶ **Starting from the collision index**, we increase the probe distance by the **square** of the number of steps

Since keys which are mapped to the same value by the primary hash function follow the **same sequence** of indexes, quadratic probing shows **secondary clustering**.

Clustering still occurs, but in different regions of the table

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	38	None	None	25	77



Open Addressing – Double hashing

- ▶ Double hashing aims to avoid both primary and secondary clustering
 - ▶ It does this by calculating a **step value** for the probing using a **second hash** function
 - ▶ In this way, the number of *skips* is dependent on the key and will be different for different keys, even if the first hash function maps them to the same position
- ▶ The second hash function, h' , should:
 - ▶ Be different from the original hash function, and
 - ▶ Not result in 0 (so that the probing progresses)
 - ▶ This is often chosen as:

$$h'(key) = q - (key \% q)$$

where q is a prime number $< n$
(and where n is the size of the
hashtable)



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

Insert 43

$$h'(key) = 7 - (key \% 7)$$

$$43 \% 13 = 4 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 43

$$43 \% 13 = 4 \text{ (collision)}$$

$$\text{Step size} = 7 - (43 \% 7) = 6$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 43

$$43 \% 13 = 4 \text{ (collision)}$$

$$\text{Step size} = 7 - (43 \% 7) = 6$$

$$(4 + 6) \% 13 = 10 \text{ (available)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 43

$$43 \% 13 = 4 \text{ (collision)}$$

$$\text{Step size} = 7 - (43 \% 7) = 6$$

$$(4 + 6) \% 13 = 10 \text{ (available)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

$$\text{Step size} = 7 - (25 \% 7) = 3$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

$$\text{Step size} = 7 - (25 \% 7) = 3$$

$$(12 + 3) \% 13 = 2 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

$$\text{Step size} = 7 - (25 \% 7) = 3$$

$$(12 + 3) \% 13 = 2 \text{ (collision)}$$

$$(12 + 6) \% 13 = 5 \text{ (collision)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

$$\text{Step size} = 7 - (25 \% 7) = 3$$

$$(12 + 3) \% 13 = 2 \text{ (collision)}$$

$$(12 + 6) \% 13 = 5 \text{ (collision)}$$

$$(12 + 9) \% 13 = 8 \text{ (available)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	43	None	77



Open Addressing – Double hashing

► Example:

$$h(key) = key \% 13$$

$$h'(key) = 7 - (key \% 7)$$

Insert 25

$$25 \% 13 = 12 \text{ (collision)}$$

$$\text{Step size} = 7 - (25 \% 7) = 3$$

$$(12 + 3) \% 13 = 2 \text{ (collision)}$$

$$(12 + 6) \% 13 = 5 \text{ (collision)}$$

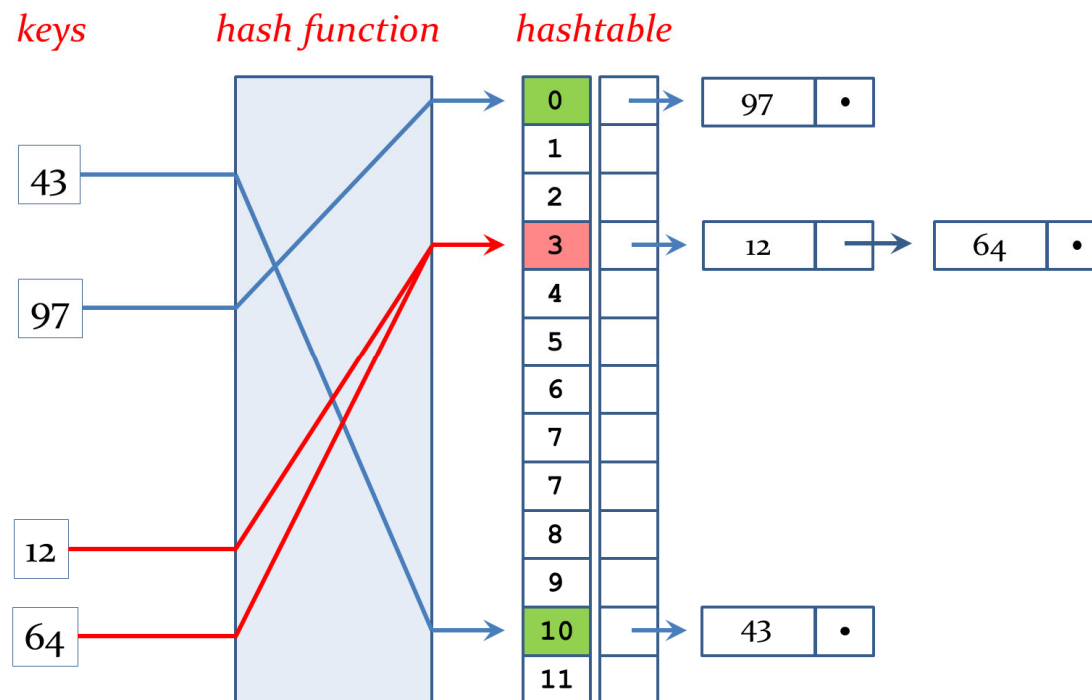
$$(12 + 9) \% 13 = 8 \text{ (available)}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	25	None	43	None	77



Collision Resolution: Separate chaining

- ▶ In **separate chaining**, the hashtable consists of a list of linked lists
 - ▶ every element of the hashtable is a linked list
 - ▶ any items which are hashed to a particular index are simply added to the corresponding list





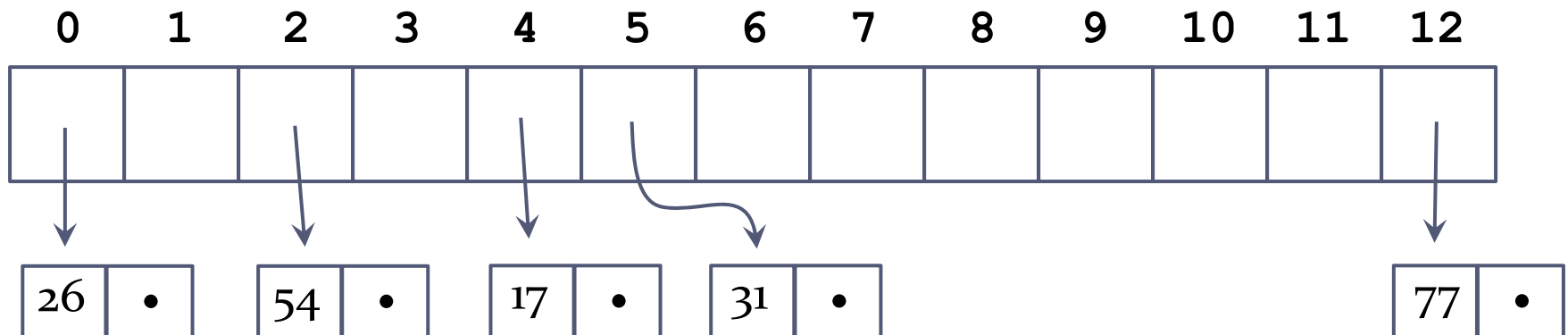
Separate chaining

► Example:

- Given the hashtable below, insert the following keys:

43 69 93 56 90

using the hash function: $h(key) = key \% 13$





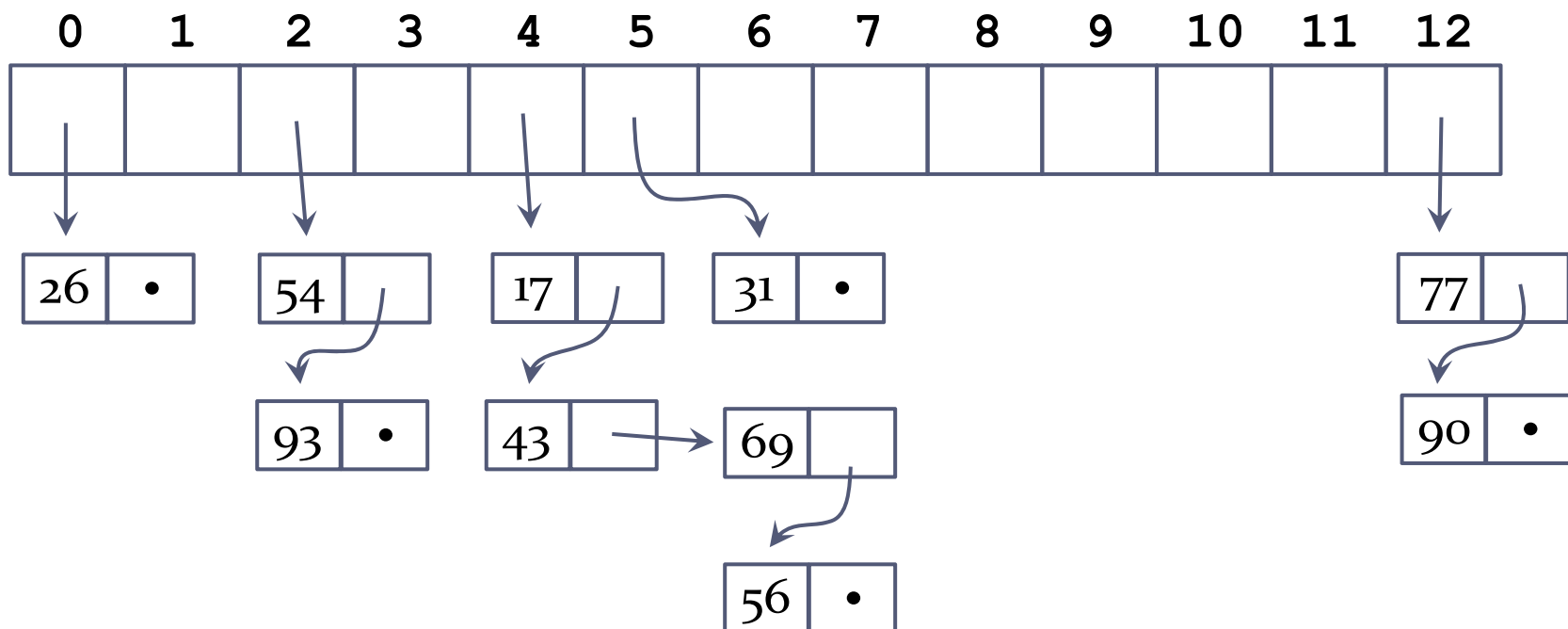
Separate chaining

► Example:

- Given the hashtable below, insert the following keys:

43 69 93 56 90

using the hash function: $h(key) = key \% 13$

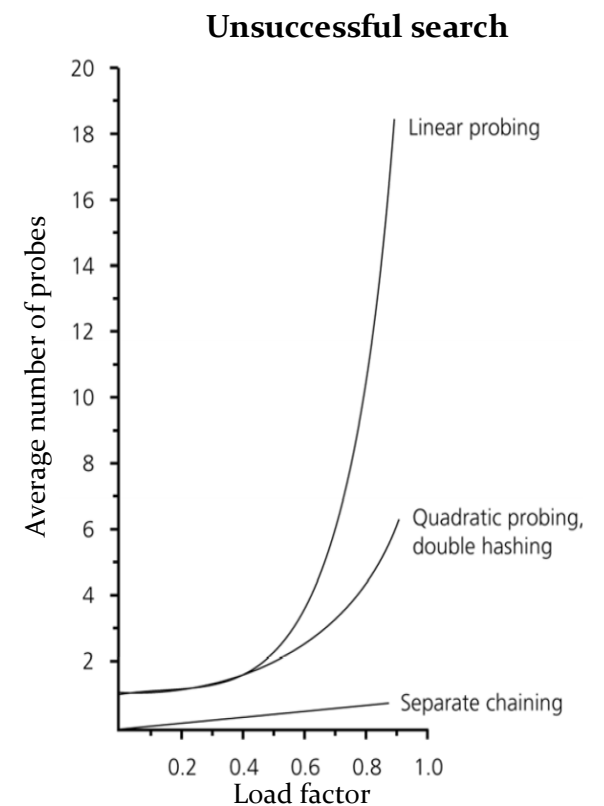
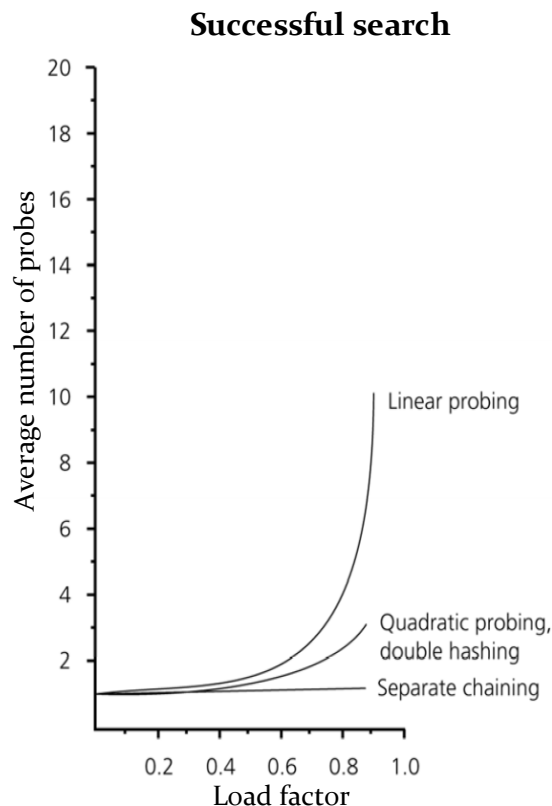




Rehashing

- ▶ Open addressing performs particularly poorly as the hashtable becomes full
 - ▶ This is not such a problem with separate chaining, as the linked lists can grow as needed (although if they get too long, the search performance will degrade)

- The **load factor** (λ) of the hashtable is the number of items in the table divided by the size of the table
- For open addressing schemes, performance degrades significantly once the load factor reaches approximately 0.7
- At this point, a new, larger, hashtable should be created
- You can't just copy the items across – they must be **rehashed** as appropriate for the new table





Exercise 2 (Linear probing)

- ▶ Consider the following hashtable:

0	1	2	3	4	5	6	7	8	9	10
88		35	25							32

- ▶ What is the hash table after inserting 10 and 11? Note: use **linear** probing in case of collision and the hash function is:

$$h(key) = key \% 11$$



Exercise 3 (Quadratic probing)

- ▶ Consider the following hashtable:

0	1	2	3	4	5	6	7	8	9	10	11	12
39	92	41			57	71			61			90

- ▶ What is the hash table after inserting 27 and 52? Note: use **quadratic** probing in case of collision and the hash function is:
$$h(key) = key \% 13$$



Exercise 4 (Double hashing probing)

- ▶ Consider the following hashtable:

0	1	2	3	4	5	6	7	8	9	10	11	12
39	92	41			57	71			61			90

- ▶ What is the hash table after inserting 31 and 53? Note: use **double hashing** probing in case of collision.
- ▶ The hash function is: $\text{key} \% 13$
- ▶ The second hash function is: $7 - (\text{key} \% 7)$.



Exercise 5 (Separate chaining)

- ▶ Insert the following items into the a hash table (size=11) that uses **separate chaining** for collision resolution:
 - ▶ 11, 25, 63, 99, 12, 35, 54, 87, 66, 75, 91

- ▶ What is the content inside the hash table?

0	1	2	3	4	5	6	7	8	9	10



Exercise 6

- ▶ Consider the following hash table:

[56, 43, 30, None, None, 26, 13]

- ▶ Rehash the above table into the hash table of size 13.
 - ▶ Note: You should use quadratic probing in case of collision and the hash function is "key % 13".

0	1	2	3	4	5	6	7	8	9	10	11	12



Exercise 7

- ▶ We have a hash table of size 13, and the following hash function:

0	1	2	3	4	5	6	7	8	9	10	11	12
26		54	94	17	31							77

- ▶ what is the **probe sequence** when inserting 44 and 51 into the hashtable? Note: use **linear** probing in case of collision and the hash function is: $\text{key} \% 13$
- ▶ **probe sequence:**
 - ▶ The list of locations which a method for open addressing produces as alternatives in case of a collision



Exercise 8

- ▶ Consider the following class definition:

```
class SimpleHashTable:
    def __init__(self, size = 7):
        self.size = size
        self.slots = [None] * size
    def __str__(self):
        return str(self.slots)
    ...
```

- ▶ Add the following methods:
 - ▶ A method named `get_hash_index(self, key, step=0)` which returns an index position
 - ▶ formula: **$(\text{key} + \text{step}) \% \text{size}$**
 - ▶ A method named `put(self, key)` method which inserts the parameter `key` into a hashtable assuming no collisions



Exercise 9

- ▶ Continuing on from the previous question, add the following methods to the SimpleHashTable class:
 - ▶ The `__len__(self)` method which returns the number of elements present in a hashtable.
 - ▶ The `get_load_factor(self)` method which returns the load factor of a hash table.
 - ▶ The load factor is the number of items in the hash table divided by the size of the table.